VERIQTA

DEVOOPS PROJECTS THAT ACTUALLY LOOK LIKE PRODUCTION

- ✓ REAL-WORLD ARCHITECTURE
- ✓ INFRASTRUCTURE AS CODE
- ✓ CI/CD PIPELINE
- ✓ MONITORING & ALERTING
- ✓ SECURITY & RELIABILITY
- ✓ INCIDENTS & POSTMORTEMS

veriqta veriqta veriqta veriqta veriqta veriqta

VERIQTA

PAGE 1

1. BUILD THE PRODUCTION ARCHITECTURE

DEFINE THE APPLICATION AND BUSINESS REQUIREMENTS

- Understand goals, features, performance and capacity needs
- Define SLAs, RTO, RPO and compliance requirements

USERS, FRONTEND, BACKEND, DATABASE

- Map user types and traffic
- Identify frontend clients,
- backend services and databases

PUBLIC VS PRIVATE COMPONENTS

- Identify internet-facing resources
- Keep sensitive components in private subnets

DNS AND ENTRY POINT

- Use Route 53 for DNS
- Define domain, records and routing policy

LOAD BALANCER

- Distribute traffic across healthy application targets
- Enable SSL/TLS

APPLICATION COMPUTE LAYER

- EC2 / Containers / ECS / EKS
- Auto Scaling for high availability
- Stateless application design

DATABASE LAYER

- Use managed DB
- Multi-AZ for high availability
- Automated backups

OBJECT STORAGE

- Store files, media, logs and backups
- Use S3 with proper encryption

CACHE

- Use ElastiCache (Redis / Memcached) for high performance and session storage

NETWORK BOUNDARIES

- VPC, Subnets, Route Tables
- Security Groups & NACLs
- Least privilege access

AVAILABILITY ZONES

- Distribute resources across multiple AZs for resilience

FAILURE DOMAINS

- AZs, subnets, instances
- Ensure no single point of failure

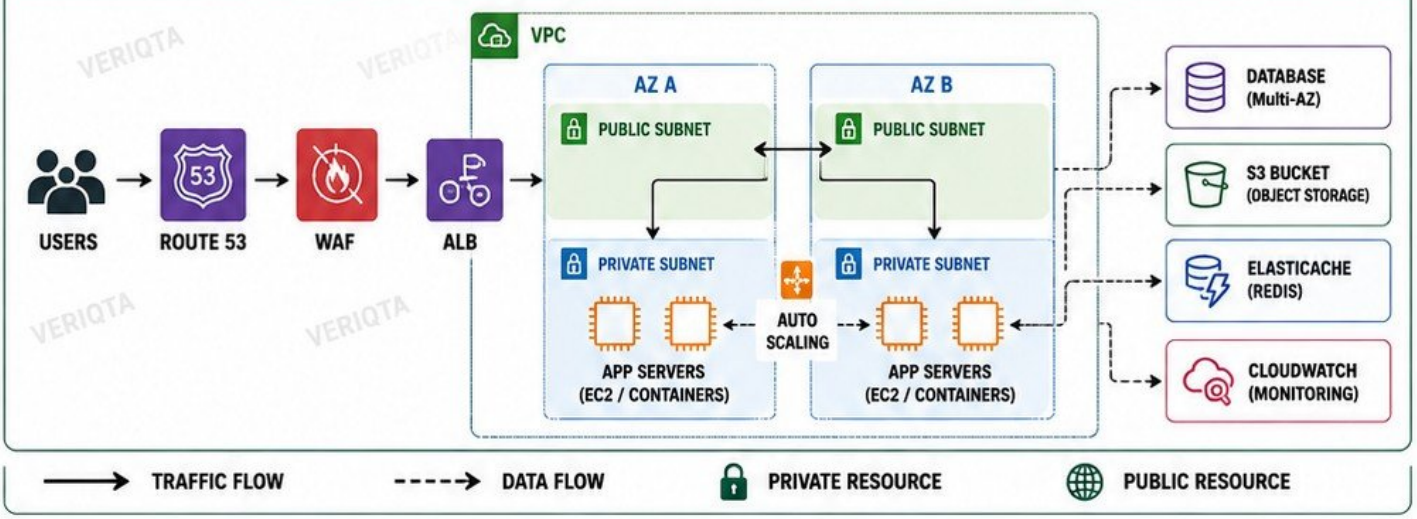
DRAW THE COMPLETE ARCHITECTURE BEFORE BUILDING

- Plan all components and their connections
- Review with stakeholders

IDENTIFY EVERY SINGLE POINT OF FAILURE

- List all dependencies
- Plan for redundancy
- Add monitoring and alerts

PROJECT ARCHITECTURE DIAGRAM (EXAMPLE)



REMEMBER: A strong architecture is planned before you build.
Good design prevents outages, reduces cost and simplifies operations.

2. BUILD THE AWS NETWORK FOUNDATION

DESIGN A SECURE, RESILIENT AND SCALABLE PRODUCTION NETWORK

GOAL

Build a highly available VPC network that isolates resources, secures traffic and supports production workloads.

- PRODUCTION VPC
- CIDR PLANNING
- MULTIPLE AZs
- PUBLIC SUBNETS
- PRIVATE APP SUBNETS
- PRIVATE DB SUBNETS
- INTERNET GATEWAY
- NAT GATEWAY
- ROUTE TABLES
- SECURITY GROUPS
- NETWORK ACL AWARENESS
- PUBLIC vs PRIVATE IP

BEST PRACTICE

Keep private resources private.

Use least privilege security rules.

Test connectivity between all layers.

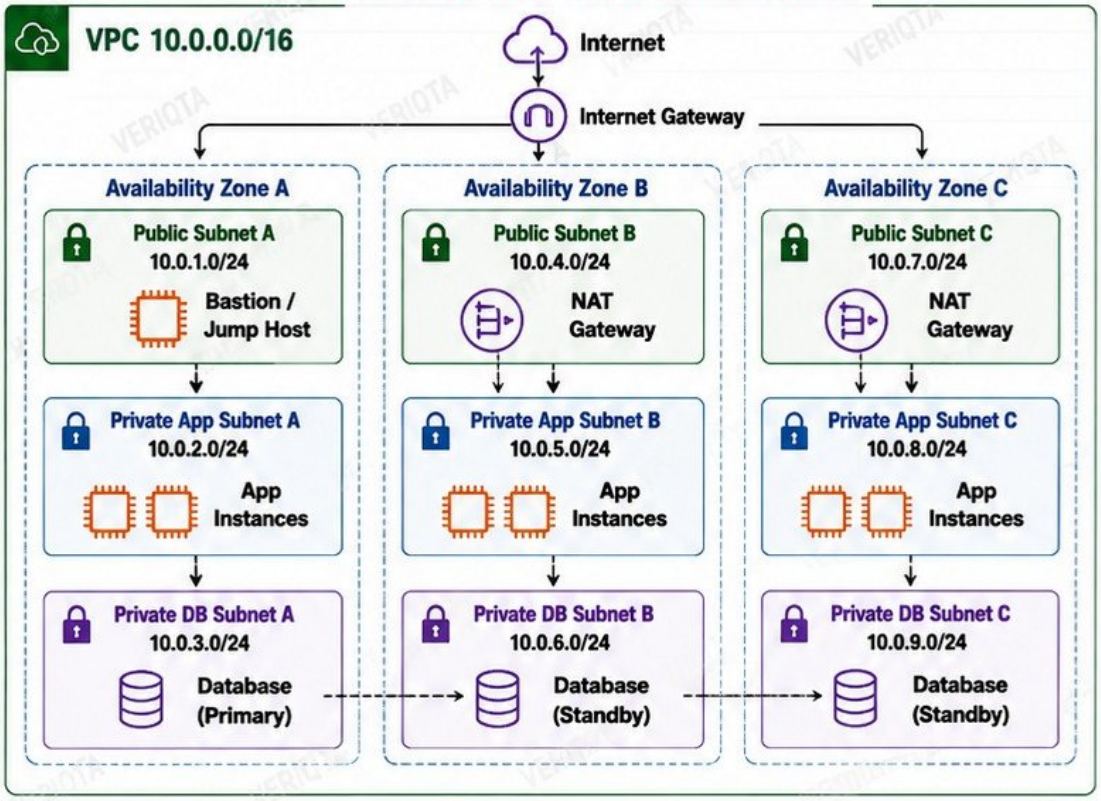
TEST CONNECTIVITY BETWEEN LAYERS

- EC2 in public subnet→ Internet
- EC2 in private app subnet → Internet (via NAT)
- App subnet → DB subnet
- DB subnet → Internet (should FAIL)
- Use ping, curl, telnet and security group checks

TROUBLESHOOT A BROKEN ROUTE

- Check route table of the affected subnet
- Verify target (IGW, NAT, VGW, VPC Endpoint)
- Confirm next hop is correct
- Check NACLs and Security Groups
- Validate source/destination IPs and ports

PRODUCTION NETWORK DIAGRAM



ROUTE TABLES

Public RT
- 0.0.0.0/0 → IGW

Private App RT
- 0.0.0.0/0 → NAT GW

Private DB RT
- Local only

SECURITY GROUPS

- Allow only required traffic
- Restrict by source IP or SG
- No open to the world

NETWORK ACLs

- Stateless
- Subnet level
- Allow / Deny rules
- Use cautiously

CODE → **BUILD** → **SCAN** → **DEPLOY**

- **CODE**: Commit code
- **BUILD**: Build container image
- **SCAN**: Scan image and infrastructure
- **DEPLOY**: Deploy to production

KEY TAKEAWAY

A strong network foundation is the backbone of secure, highly available AWS production workloads.

3. PROVISION EVERYTHING WITH TERRAFORM

Define. Provision. Version. Repeat.

TERRAFORM PROJECT STRUCTURE

```
terraform-project/
  providers.tf
  variables.tf
  locals.tf
  outputs.tf
modules/
  vpc/
  compute/
  database/
environments/
  vpc/
  staging/
  prod/
terraform.tfvars
backend.tf
```

PROVIDERS
Define the cloud or platform provider (e.g., AWS, Azure, GCP).

VARIABLES
Make configurations reusable and environment-aware.

LOCALS
Store expressions or values used across the configuration.

OUTPUTS
Expose values after provisioning (e.g., endpoints, IDs, ARNs).

REUSABLE MODULES
Organize infrastructure into reusable building blocks.

REMOTE STATE
Store Terraform state remotely for consistency and team collaboration.

S3 STATE STORAGE
Use an S3 bucket to store the state file securely.

STATE LOCKING
Prevent concurrent updates using DynamoDB locking.

ENVIRONMENT SEPARATION
DEV — STAGING — PROD

Separate workspaces or directories for isolated environments.

VPC MODULE
Create VPC, subnets, route tables, IGW, NAT gateways, etc.

COMPUTE MODULE
Provision EC2, ASG, Launch Templates, Security Groups, IAM roles, etc.

DATABASE MODULE
Provision RDS, subnets, security groups, backups, parameter groups, etc.

TERRAFORM CLI WORKFLOW

terraform fmt
Format code for consistency.

terraform validate
Check configuration syntax and validity.

terraform plan
Preview changes before applying.

terraform apply
Apply the changes and provision infrastructure.

INFRASTRUCTURE CHANGE WORKFLOW

1. **WRITE CODE** — Define infrastructure as code.
2. **PLAN** — Review planned changes.
3. **APPLY** — Provision or update infrastructure.
4. **VERIFY** — Validate resources and outputs.

AVOID MANUAL INFRASTRUCTURE DRIFT

- No manual changes in console
- Everything defined in code
- Version controlled
- Repeatable and auditable
- Safe, consistent and scalable

Terraform ensures your infrastructure is consistent, repeatable and version controlled. Code is the source of truth.

4. BUILD A PRODUCTION GIT WORKFLOW

1. REPOSITORIES

APPLICATION REPOSITORY

Contains your application code.

INFRASTRUCTURE REPOSITORY

Contains IaC and environment configurations.

2. BRANCH PROTECTION

- Protect main, staging, production branches
- Require pull requests
- Require status checks to pass
- Require code review
- Prevent force push
- Prevent branch deletion

3. FEATURE BRANCHES

Create a branch from main for every new feature or fix.

4. PULL REQUESTS

Open a pull request to merge your changes into the target branch.

5. CODE REVIEW

At least one qualified reviewer must approve the changes.

6. REQUIRED STATUS CHECKS

All status checks must pass before merging (tests, linting, security scans, build).

7. CODEOWNERS

Define owners for paths and require their review on changes.

8. COMMIT DISCIPLINE

- Use meaningful commit messages
- Make small, focused commits
- One logical change per commit
- Follow team's commit message format

9. RELEASE TAGS

Tag every release in the repository (e.g., v1.2.0).

10. SEMANTIC VERSIONING

MAJOR . MINOR . PATCH

1 . 2 . 3

- MAJOR: Breaking changes
- MINOR: New features
- PATCH: Bug fixes

11. PREVENT DIRECT PRODUCTION CHANGES

Never push directly to production or main. Always go through pull requests.

12. HANDLE A REJECTED PULL REQUEST

- Read reviewer feedback carefully
- Update your branch
- Address all comments
- Request review again
- Never ignore feedback

13. ROLL BACK A BAD CHANGE

- Identify the last good release/tag
- Revert the commit or merge
- Deploy the previous good state
- Document what happened

14. GIT AS THE SOURCE OF CHANGE HISTORY

- Every change is versioned
- Full audit trail of who, what, when, why
- Essential for debugging, compliance and rollbacks

KEY TAKEAWAY

A strong Git workflow protects your production environment, ensures quality, enables safe collaboration and gives you complete control over every change.

CODE → REVIEW → MERGE → TAG → DEPLOY

5. CONTAINIERIZE THE APPLICATION PROPERLY

Build secure, efficient and production-ready container images.

1. PRODUCTION DOCKERFILE

```
# Multi-stage build example
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:20-alpine AS production
WORKDIR /app
ENV NODE_ENV=production
COPY --from=builder /app/dist ./dist
COPY package*.json ./
RUN npm ci --omit=dev
USER node
EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=3s \
  CMD wget -qO- http://localhost:3000/health || exit 1
CMD ["node", "dist/server.js"]
```

2. MULTI-STAGE BUILDS

Build in one stage, run in another. Keep only what you need.

3. SMALL BASE IMAGES

Use minimal images like Alpine, Distroless or Slim.

4. DEPENDENCY MANAGEMENT

Install only production dependencies. Avoid bloat.

5. NON-ROOT CONTAINERS

Run as non-root user for better security.

6. .DOCKERIGNORE

.dockerignore

Exclude files and folders you don't need in the image build.

7. ENVIRONMENT CONFIGURATION

Use ENV vars. Never hardcode secrets or configs.

8. HEALTH CHECKS

Define health checks so orchestrators know when your app is healthy.

9. IMAGE TAGGING

Use meaningful tags. Example: v1.2.0, 2.3.1 not latest.

10. IMMUTABLE IMAGES

Never change an image. Build a new image for every change.

11. VULNERABILITY SCANNING

Scan images for vulnerabilities before pushing to production.

12. BUILD AND INSPECT THE IMAGE

BUILD THE IMAGE

```
docker build -t myapp:1.0 .
```

INSPECT THE IMAGE

```
docker images myapp:1.0
docker inspect myapp:1.0
```

ANALYZE LAYERS AND SIZE

```
docker history myapp:1.0
docker image ls -s
```

13. RUN LOCALLY

```
docker run -d --name myapp \
-p 3000:3000 myapp:1.0
```

```
docker ps
docker logs myapp
```

14. DEBUG A CONTAINER THAT IMMEDIATELY EXITS

1. Check logs
   ```
   docker logs myapp
   ```
2. Run interactively
   ```
   docker run -it --rm myapp:1.0 sh
   ```
3. Check entrypoint, command, env vars and ports.

15. REDUCE UNNECESSARY IMAGE SIZE

- Use minimal base images
- Use multi-stage builds
- Remove dev dependencies
- Clean package caches
- Exclude files with .dockerignore
- Verify image size before pushing

THE RIGHT WORKFLOW

SCAN → FIX → REBUILD → RESCAN

WHY REBUILDING MATTERS

Changes in code or dependencies are not reflected in the old image.

Rebuild ensures every fix is actually included in the final image.

6. BUILD A CONTAINER REGISTRY WORKFLOW

Build, store and deliver container images securely and reliably.

1. **CREATE THE IMAGE REPOSITORY**
   Amazon ECR or equivalent registry.
2. **REGISTRY AUTHENTICATION**
   Authenticate your client with the registry securely.
3. **BUILD IMAGE**
   Build your container image from Dockerfile.
4. **TAG IMAGE**
   Tag the image with meaningful, versioned tags.
5. **SCAN IMAGE**
   Scan the image for vulnerabilities before pushing.
6. **PUSH IMAGE**
   Push the image to the container registry.

7. VERSIONED IMAGE TAGS

- Use versioned tags for traceability and rollback.
- Examples:
  - v1.0.0
  - v1.2.3
  - 1.2.3
  - 2024-05-26
  - build-123

8. COMMIT SHA TAGGING

`abc1234`

Always tag images with the commit SHA for exact traceability.

Example: abc1234

9. AVOID RELYING ON LATEST

latest is mutable and can break deployments.
Always use specific tags.

10. IMAGE RETENTION POLICIES

- Keep only necessary versions.
- Delete untagged images.
- Use lifecycle policies (ECR lifecycle rules).
- Reduce storage costs.
- Retain images needed for rollback and compliance.

11. PULL IMAGE INTO ANOTHER ENVIRONMENT

- Authenticate with the registry.
- Pull the specific image tag.
- Verify image integrity.
- Deploy to target environment.

12. HANDLE AUTHENTICATION FAILURE

- Check credentials and permissions.
- Verify token expiration.
- Ensure correct region and registry URL.
- Re-authenticate and retry.

13. HANDLE MISSING IMAGE TAGS

- Verify the tag name.
- Check case sensitivity.
- Confirm tag was pushed.
- List available tags in registry.

14. TRACE DEPLOYMENT BACK TO SOURCE COMMIT

- Use commit SHA tagging.
- Record image tag in deployment manifests.
- Link deployment → image → commit.
- Enables debugging and audits.

WORKFLOW SUMMARY

CREATE REPOSITORY → AUTHENTICATE → BUILD → TAG → SCAN → PUSH → PULL & DEPLOY

A STRONG REGISTRY WORKFLOW = SECURE, TRACEABLE, AND RELIABLE DEPLOYMENTS

7. BUILD CI THAT DEVELOPERS CAN TRUST

FAST • RELIABLE • REPRODUCIBLE • SECURE

1. CI PIPELINE FLOW (HIGH LEVEL)

1. **PULL REQUEST TRIGGER** — Developer opens or updates a PR
2. **DEPENDENCY INSTALLATION** — Install all required packages and tools
3. **LINTING** — Check code style and quality
4. **UNIT TESTS** — Run fast, isolated unit tests
5. **INTEGRATION TESTS** — Test components together
6. **BUILD VALIDATION** — Ensure build is successful
7. **DOCKER IMAGE BUILD** — Build container image
8. **SECURITY SCANNING** — Scan image, code, deps and IaC
9. **ARTIFACT GENERATION** — Generate reports, binaries, artifacts
10. **PIPELINE CACHING** — Cache deps, layers and build outputs
11. **PARALLEL JOBS** — Run jobs in parallel to save time
12. **PIPELINE FAILURE HANDLING** — Fail fast, notify and collect logs

2. REQUIRED CHECKS BEFORE MERGE

- All jobs completed successfully
- No linting errors
- All unit tests passed
- All integration tests passed
- Build validation passed
- Security scans passed
- No critical or high vulnerabilities
- Artifacts generated
- Code review approved
- Up-to-date with target branch

3. PIPELINE PERFORMANCE BOOSTERS

PIPELINE CACHING

- Cache dependencies
- Cache Docker layers
- Cache build outputs
- Speeds up repeat runs

PARALLEL JOBS

- Run independent jobs in parallel
- Tests, lint, scan
- Reduce total time

OPTIMIZED AGENTS

- Right size runners
- Use faster machines
- Keep tools updated

4. READ PIPELINE LOGS SYSTEMATICALLY

1. **OPEN THE FAILED JOB** — Check the job that failed first.
2. **READ FROM TOP TO BOTTOM** — Start from the beginning of the logs.
3. **FIND THE FIRST ERROR** — Scroll up to the first actual error.
4. **READ THE ERROR MESSAGE CAREFULLY** — Understand what failed and why.
5. **CHECK THE CONTEXT** — Look at commands, inputs and environment.
6. **IDENTIFY THE ROOT CAUSE** — Focus on the real cause, not the last line.
7. **TEST THE FIX LOCALLY** — Reproduce locally and confirm the fix.
8. **RE-RUN AND VERIFY** — Push the fix and confirm the pipeline passes.

5. DIAGNOSE TEST THAT PASSES LOCALLY BUT FAILS IN CI

- **CHECK ENVIRONMENT DIFFERENCES** — OS, Node/Python version, databases, services, env variables.
- **CHECK DEPENDENCIES** — Different versions can cause failures.
- **CHECK TEST DATA** — Data setup may differ in CI.
- **CHECK TIMING AND PARALLELISM** — Tests that depend on order may fail in CI.
- **CHECK EXTERNAL SERVICES** — APIs, databases or third-party services.
- **REPRODUCE CI ENVIRONMENT LOCALLY** — Use Docker or the same CI image.
- **ADD LOGS AND DEBUG OUTPUT** — Print variables, states and responses.
- **FIX THE ROOT CAUSE, NOT THE SYMPTOM** — Avoid hardcoded values or workarounds.

A GOOD CI PIPELINE GIVES DEVELOPERS CONFIDENCE. EVERY COMMIT, EVERY TIME.

8. BUILD THE CONTINUOUS DELIVERY PIPELINE

ONE BUILD. ONE ARTIFACT. MANY ENVIRONMENTS.

01	MERGE TO MAIN	Code is merged to the main branch after review and automated checks.
02	BUILD ONCE	The application is built one time in a clean, reproducible environment.
03	VERSION ARTIFACT	The artifact is versioned using semantic versioning or build metadata.
04	PUSH TO REGISTRY	The versioned artifact (container image) is pushed to the container registry.
05	DEPLOY TO DEVELOPMENT	Deploy the exact artifact to the development environment.
06	AUTOMATED VALIDATION	Run automated tests (unit, integration, security, and quality checks).
07	PROMOTE TO STAGING	Promote the same artifact to the staging environment (no rebuild).
08	SMOKE TESTS	Execute smoke tests and basic validations to confirm stability.
09	PRODUCTION APPROVAL	Manual approval from the right stakeholder before production deployment.
10	PRODUCTION DEPLOYMENT	Deploy the exact same artifact to production using automated deployment.
11	DEPLOYMENT VERIFICATION	Verify health, availability, logs, metrics and business functionality.
12	NOTIFICATIONS	Notify the team about successful deployments and important events.
13	FAILED DEPLOYMENT HANDLING	Detect failure quickly and trigger alerts, stop bad deployments, and protect users.
14	ROLLBACK WORKFLOW	Rollback to the last known good version using a tested rollback process.

PIPELINE PRINCIPLES

RELIABILITY
Automate checks at every stage.

SECURITY
Scan and validate before promotion.

CONSISTENCY
The same artifact moves through all environments.

SPEED
Fast feedback, fast recovery, zero guesswork.

NEVER REBUILD AN ARTIFACT BETWEEN ENVIRONMENTS.

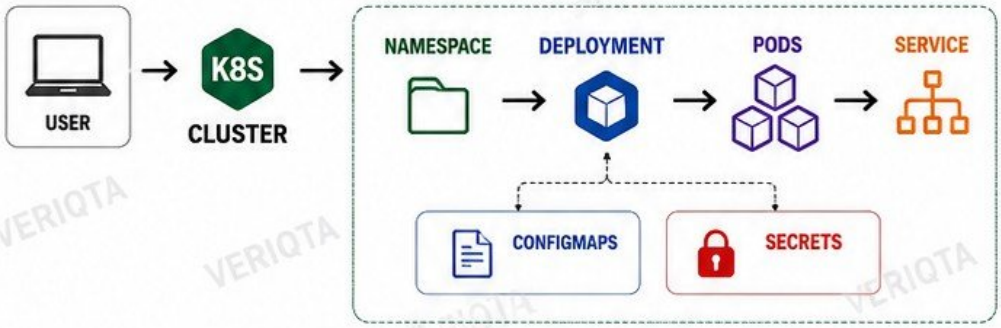
KEY TAKEAWAY

Build once, test everywhere, promote with confidence, and deliver value to users safely and consistently.

9. DEPLOY THE APPLICATION TO KUBERNETES

Deploy, manage and operate production-ready applications on Kubernetes.

KUBERNETES ARCHITECTURE OVERVIEW



CORE K8S OBJECTS

NAMESPACE
Logical isolation for resources.

DEPLOYMENT
Manages replica sets and updates.

PODS
Smallest deployable units that run containers.

SERVICE
Stable network endpoint to access pods.

CONFIGMAPS
Store non-sensitive configuration data.

SECRETS
Store sensitive data such as passwords, tokens and keys.

RESOURCE MANAGEMENT

RESOURCE REQUESTS

Minimum resources the container needs.

```
resources:
  requests:
    cpu: "250m"
    memory: "256Mi"
```

RESOURCE LIMITS

Maximum resources the container can use.

```
resources:
  limits:
    cpu: "500m"
    memory: "512Mi"
```

WHY IT MATTERS

- Prevents noisy neighbors
- Better scheduling
- Avoids OOMKills
- Improves stability

HEALTH CHECKS (PROBES)

LIVENESS PROBE
Checks if the container is alive. Restarts the container if it fails.

```
livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
```

READINESS PROBE
Checks if the container is ready to receive traffic.

```
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
```

STARTUP PROBE
Gives slow-starting containers time to initialize.

```
startupProbe:
  httpGet:
    path: /startup
    port: 8080
```

UPDATES & AVAILABILITY

ROLLING UPDATES
Update pods gradually without downtime.

REPLICA STRATEGY
Maintain the desired number of pod replicas.

POD DISRUPTION AWARENESS
Distribute pods across nodes and zones to maintain availability.

DEPLOY & INSPECT WORKLOADS

- **Apply:** `kubectl apply -f deployment.yaml`
- **Get Pods:** `kubectl get pods -n <namespace>`
- **Describe:** `kubectl describe pod <pod-name>`
- **Logs:** `kubectl logs <pod-name>`
- **Exec:** `kubectl exec -it <pod-name> -- /bin/sh`

DEBUG COMMON FAILURES

CrashLoopBackOff
Container crashes repeatedly.
- Check logs: kubectl logs <pod-name>
- Fix the application or configuration.

ImagePullBackOff
Kubernetes cannot pull the image.
- Check image name and registry.
- Verify image pull secrets and access.

KEY TAKEAWAY

Deploy with best practices, monitor your workloads, and debug issues fast to keep applications reliable in production.

10. EXPOSE THE APPLICATION SAFELY

KEY CONCEPTS

INTERNAL vs EXTERNAL SERVICES
Internal services are only accessible inside the cluster. External services are reachable from outside.

INGRESS
Kubernetes resource that defines rules for external access to services.

INGRESS CONTROLLER
Software that watches Ingress resources and configures the proxy/load balancer.

CLOUD LOAD BALANCER
Distributes traffic across nodes running the Ingress Controller (usually a Service of type LoadBalancer).

DNS
Maps your domain name to the public IP or hostname of the load balancer.

TLS / HTTPS
Encrypts traffic in transit to protect users and data.

CERTIFICATE MANAGEMENT
Use cert-manager or cloud provider tools to issue and renew TLS certificates.

HOST-BASED ROUTING
Route traffic based on the domain name (e.g. app.example.com).

PATH-BASED ROUTING
Route traffic based on URL path (e.g. /api, /web).

SECURITY GROUPS
Control inbound and outbound traffic to allow only what is necessary.

END-TO-END REQUEST FLOW

USER → **DNS** → **LOAD BALANCER** → **INGRESS** → **SERVICE** → **POD**

- **DNS**: Resolves domain to Load Balancer IP
- **LOAD BALANCER**: Receives external traffic and forwards to Ingress Controller
- **INGRESS**: Applies rules and routes traffic to the correct Service
- **SERVICE**: Routes traffic to the appropriate Pods (endpoints)
- **POD**: Application processes the request and returns response

RESPONSE RETURNS THE SAME PATH

DEBUG A 502 RESPONSE

1. **CHECK POD HEALTH**
   Ensure Pods are Running and Ready.
   `kubectl get pods -n <namespace>`
2. **CHECK SERVICE ENDPOINTS**
   Ensure the Service has healthy endpoints.
   `kubectl get endpoints -n <namespace> <service>`
3. **CHECK INGRESS CONTROLLER LOGS**
   Look for upstream errors or connection issues.
   `kubectl logs -n <ingress-namespace> <ingress-pod>`
4. **CHECK BACKEND POD LOGS**
   Verify the application is not crashing or returning errors.
5. **CHECK TIMEOUT SETTINGS**
   Increase timeouts if your application takes longer to respond.
6. **CHECK SECURITY GROUPS / NETWORK**
   Ensure traffic from Load Balancer to Nodes is allowed.

APP WORKS INSIDE CLUSTER BUT NOT EXTERNALLY

1. **CHECK DNS RESOLUTION**
   Does the domain resolve to the correct Load Balancer IP?
2. **VERIFY LOAD BALANCER HEALTH**
   Is the Load Balancer healthy and targeting the nodes?
3. **CHECK INGRESS RULES**
   Host/path rules may be incorrect or missing.
4. **VERIFY SERVICE TYPE**
   Ensure the Service is ClusterIP (for Ingress) and exposed correctly.
5. **CHECK SECURITY GROUPS / FIREWALL**
   Ensure inbound 80/443 allowed to Load Balancer and NodePort (if used).
6. **CHECK TLS / CERTIFICATE**
   Certificate mismatch or expired cert can break HTTPS.
7. **CHECK PROXY / APPLICATION CONFIG**
   Ensure the app is aware of correct host/protocol headers.

KEY TAKEAWAYS

- Expose only what is necessary.
- Use TLS everywhere.
- Use Ingress for flexible routing.
- Keep Security Groups tight.
- Monitor and check logs.
- Test from outside like a real user.
- Follow the end-to-end path when debugging.
- A 502 usually means upstream or connectivity issues.

11. ADD PRODUCTION CONFIGURATION AND SECRETS

CONFIGURATION vs SECRETS

CONFIGURATION

- Non-sensitive settings like URLs, feature flags, timeouts, log levels.
- Can be public.

VS

SECRETS

- Sensitive data like passwords, API keys, tokens, certificates.
- Must be protected, encrypted and access controlled.

ENVIRONMENT-SPECIFIC CONFIGURATION

- Different values for Dev, Staging and Production.
- Use environment variables or configuration systems.
- Keep code the same, change configuration.

KUBERNETES CONFIGMAPS

- Store non-sensitive configuration.
- Key-value pairs.
- Mounted as files or environment variables.

KUBERNETES SECRETS

- Store sensitive data (Base64 encoded).
- Used as environment variables or files.
- Access controlled by RBAC.

EXTERNAL SECRET STORES

Use managed secret stores for better security, auditing and rotation.

AWS SECRET STORES

AWS SECRETS MANAGER

- Store, rotate and manage secrets.
- Automatic rotation support.
- Fine-grained IAM access.

AWS PARAMETER STORE

- Store configuration and secrets as SecureString.
- Hierarchical naming.
- Lower cost for simple needs.

SECRET INJECTION

- Inject secrets into Pods at runtime.
- Use environment variables or mounted files.
- Avoid baking secrets into images.

IAM-BASED ACCESS

- Grant least privilege IAM permissions.
- Workload identity for Pods and services.
- Audit access with CloudTrail.

SECRET ROTATION

- Rotate secrets regularly.
- Reduce risk if a secret is exposed.
- Test new secret before revoking the old one.

AVOID CREDENTIALS IN GIT

- Never commit secrets or keys.
- Use .gitignore.
- Scan for secrets before commit.

AVOID SECRETS IN DOCKER IMAGES

- Do not store secrets in Dockerfile or image layers.
- Use runtime injection instead.
- Images are shared and cached.

HANDLE MISSING CONFIGURATION

- Application should fail fast.
- Clear error messages.
- Use defaults only for non-sensitive values.

HANDLE AN EXPIRED CREDENTIAL

- Detect expiration early.
- Implement automatic refresh or rotation.
- Alert and retry securely.

REMOVE AN ACCIDENTALLY COMMITTED SECRET SAFELY

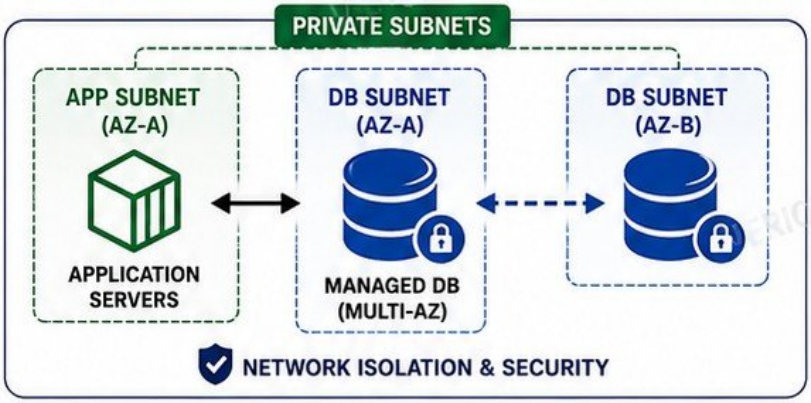
1. Rotate or revoke the exposed secret immediately.
2. Remove the secret from code history (BFG Repo-Cleaner or filter-repo).
3. Force push the cleaned history.
4. Invalidate caches and CI/CD artifacts.
5. Notify your team and audit access logs.

KEY TAKEAWAY: PROTECT CONFIGURATION AND SECRETS EVERYWHERE. STORE SAFELY, ACCESS SECURELY, ROTATE REGULARLY.

12. BUILD THE DATABASE LAYER

1. DATABASE ARCHITECTURE

- Managed relational database (AWS RDS / Aurora / Cloud SQL)
- Private database subnet
- Application-to-database connectivity
- Multi-AZ deployment for high availability



2. SECURITY

- Database security groups (allow only required ports from application layer)
- Strong credentials (no hardcoded secrets)
- Use Secrets Manager / Parameter Store
- Encrypt in transit (TLS) and at rest

3. CONNECTION MANAGEMENT

- Use secure connection strings
- Enable connection pooling
- Set max connections appropriately
- Monitor connection usage
- Prevent connection exhaustion

4. SCHEMA & MIGRATIONS

- Use version-controlled migrations
- Automate schema migrations
- Test migrations in staging
- Avoid manual schema changes in production

5. BACKUPS & RECOVERY

- Enable automated backups
- Configure backup retention
- Use point-in-time recovery (PITR)
- Test restore regularly
- Store backups in secure location

6. HIGH AVALABILITY

- Multi-AZ deployment enabled
- Automatic failover configured
- Monitor replication lag
- Test failover behavior periodically

7. ENCRYPTION

- Encryption at rest (KMS)
- Encryption in transit (TLS)
- Manage keys securely
- Rotate keys when required

8. MONITORING

- Enable database monitoring
- Track CPU, memory, storage, IOPS
- Set up alerts for key metrics
- Integrate with CloudWatch / Monitoring tools

9. PERFORMANCE

- Monitor slow queries
- Use indexes effectively
- Analyze query performance
- Optimize regularly

10. COMMON ISSUES

- Slow queries
- Connection exhaustion
- High CPU / memory
- Lock contention
- Replication lag

11. RESILIENCE & INCIDENT HANDLING

- Simulate database unavailability
- Failover to standby (Multi-AZ)
- Restore from backup if needed
- Verify application connectivity
- Recover without blindly restarting everything
- Communicate and validate data integrity

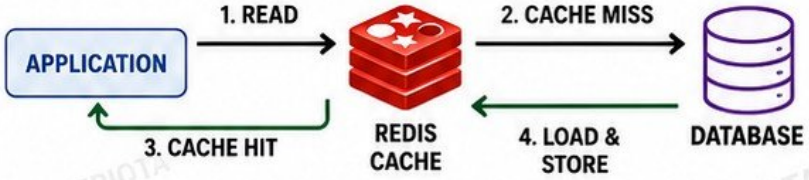
KEY TAKEAWAY: A secure, highly available and well-monitored database layer is critical for application reliability and data durability.

13. ADD CACHING AND ASYNCHRONOUS PROCESSING

REDIS CACHE

- Fast in-memory key-value store
- Great for reducing database load and improving response time
- Used for sessions, tokens, metadata, query results, etc.

CACHE-ASIDE PATTERN



CACHE HIT vs MISS

CACHE HIT
Data found in Redis.

CACHE MISS
Data not in cache. Fetch from DB, then store in Redis.

TTL (TIME TO LIVE)

Each key expires after a set time to keep data fresh.

CACHE INVALIDATION

Remove or expire stale data when underlying data changes.

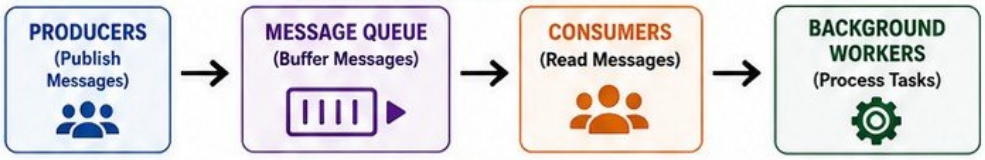
BEST PRACTICES

- Use appropriate TTL
- Invalidate on updates
- Avoid caching sensitive data
- Monitor hit ratio

MESSAGE QUEUE

Enables asynchronous processing and decouples services.

HOW IT WORKS



PRODUCERS

Send messages to the queue.

CONSUMERS

Read messages from the queue.

BACKGROUND WORKERS

Process messages asynchronously.

RETRIES

Retry failed messages automatically.

DEAD-LETTER QUEUE (DLQ)

Stores messages that continuously fail after max retries.

IDEMPOTENCY

Ensure operations can be processed multiple times safely.

QUEUE DEPTH

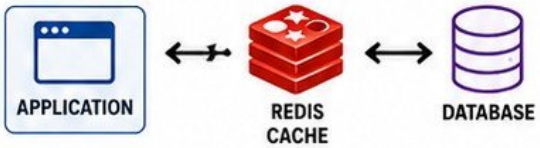
Number of messages waiting. High depth may indicate backlog.

POISON MESSAGES

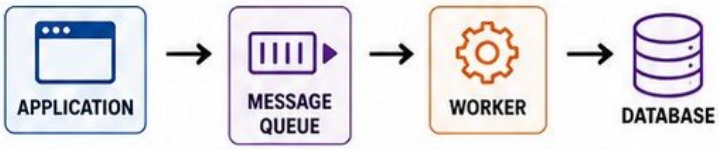
Bad messages that always fail. Send to DLQ to prevent blocking the queue.

ARCHITECTURE OVERVIEW

APPLICATION → REDIS



APPLICATION → QUEUE → WORKER



TROUBLESHOOT: GROWING QUEUE BACKLOG

- Check worker health and number of active workers.
- Check message processing time.
- Check for slow downstream services or DB.
- Increase workers or auto-scaling.
- Check for poison messages (look at DLQ).
- Monitor queue depth and set alerts.

WHAT HAPPENS WHEN REDIS DISAPPEARS?

- All cache lookups fail.
- Applications fall back to DB (cache miss).
- Increased DB load and higher response time.
- Sessions may be lost (if stored in cache).
- Temporary failures until Redis returns.
- Design apps to be cache-resilient (graceful fallback).

KEY TAKEAWAY

Caching makes things fast. Queues make things reliable.
Use both together to build scalable, resilient applications.

14. BUILD

FULL OBSERVABILITY

OBSERVE EVERYTHING. UNDERSTAND FAST. FIX WITH CONFIDENCE.

METRICS

Numbers that show the health and performance.

- Application metrics
- Infrastructure metrics
- Kubernetes metrics
- Request latency
- Error rate
- Traffic
- Saturation

LOGS

Detailed events that explain what happened.

- Centralized logging
- Structured logs
- Log aggregation
- Search and filter
- Retain and archive
- Alert on important logs

TRACES

Follow a request as it moves across services.

- Distributed tracing
- Service map
- Span timeline
- Latency breakdown
- Bottleneck detection
- Root cause analysis

KEY OBSERVABILITY TOOLS

PROMETHEUS
Metrics collection and alerting.

GRAFANA
Dashboards and visualization.

CLOUDWATCH
AWS native metrics, logs and alerts.

KUBERNETES METRICS
Cluster, node, pod and workload metrics.

OBSERVABILITY ESSENTIALS

CORRELATION IDS
Track a single request across all services.

USEFUL DASHBOARDS
Show what matters. Clear, simple and actionable.

ACTIONABLE ALERTS
Alert on symptoms that indicate real problems.

REDUCE NOISE
Focus on high signal alerts, not too many false alarms.

BUSINESS CONTEXT
Understand impact on users, revenue and systems.

SPEED TO INSIGHT
Find the issue fast. Fix faster.

FOLLOW ONE FAILED REQUEST ACROSS THE SYSTEM

USER → Request received

→ **API GATEWAY** → Route and authenticate

→ **SERVICE A** → Process request

→ **SERVICE B** → Call downstream service

→ **DATABASE** → Read / Write data

→ **ERROR** → Failure occurs here

TRACES + LOGS + METRICS + CORRELATION ID = ROOT CAUSE FOUND

FULL OBSERVABILITY = SEE EVERYTHING, UNDERSTAND EVERYTHING, FIX FASTER.
BETTER INSIGHT. BETTER RELIABILITY. BETTER SYSTEMS.

VERIQTA

PAGE 15

15. ENGINEER RELIABILITY AND SELF-HEALING

BUILD SYSTEMS THAT HEAL THEMSELVES, NOT SYSTEMS THAT BREAK AND ALERT.

CORE RELIABILITY PATTERNS

HORIZONTAL POD AUTOSCALER
Automatically scales pods based on CPU, memory or custom metrics.

REPLICA STRATEGY
Run multiple replicas of your application to ensure high availability.

POD ANTI-AFFINITY
Spread pods across different nodes to avoid single node failure.

TOPOLOGY SPREAD
Distribute pods evenly across zones, nodes or other topology domains.

MULTIPLE AVAILABILITY ZONES
Run workloads across multiple AZs to survive a zone failure.

POD DISRUPTION BUDGETS
Limit the number of pods that can be unavailable during disruptions.

HEALTH PROBES
Liveness, readiness and startup probes ensure only healthy pods receive traffic.

GRACEFUL TERMINATION
Give applications time to finish work before a pod is stopped.

ROLLING UPDATES
Update pods gradually to prevent downtime and allow easy rollback.

RETRY BEHAVIOR
Retry failed requests with backoff to handle temporary failures.

TIMEOUTS
Set timeouts to prevent hanging requests from consuming resources.

CIRCUIT BREAKERS
Stop sending requests to failing dependencies to protect your system.

DEPENDENCY FAILURES
Design for failures. Assume dependencies will fail.

CHAOS TEST YOUR SYSTEM

REMOVE ONE POD INTENTIONALLY	Observe how the system recreates and restores it.
REMOVE ONE NODE INTENTIONALLY	Observe pod rescheduling and traffic continuity.

SELF-HEALING WORKFLOW

DETECT → Health probes, metrics and alerts detect an issue.

ISOLATE → Kubernetes removes unhealthy pods from service.

REPLACE → New pods are scheduled on healthy nodes.

RECOVER → Traffic resumes automatically. System stays available.

LEARN & IMPROVE → Review, improve configuration and eliminate weak points.

IDENTIFY REMAINING SINGLE POINTS OF FAILURE

- Single database instance
- No multi-AZ deployment
- Stateful workloads without backups
- Critical dependencies without failover
- External APIs without fallbacks
- Shared storage or network bottlenecks

16. SECURE THE DELIVERY AND RUNTIME ENVIRONMENT

SECURE BY DESIGN. SECURE BY DEFAULT. VERIFY ALWAYS.

LEAST PRIVILEGE

- Grant only the access required
- Remove unnecessary permissions
- Review regularly

IAM ROLES

- Use IAM roles, not long-term keys
- Scope roles narrowly
- Rotate and audit regularly

WORKLOAD IDENTITY

- Use OIDC / IRSA for Kubernetes
- No static credentials in workloads
- Short-lived tokens

CI/CD PERMISSIONS

- Minimal pipeline permissions
- Separate build and deploy roles
- Isolate environments

DEPENDENCY SCANNING

- Scan dependencies early and often
- Catch vulnerable libraries
- Keep dependencies up to date

SECRET SCANNING

- Detect secrets in code and configs
- Prevent secret leakage
- Rotate exposed secrets

CONTAINER IMAGE SCANNING

- Scan images for vulnerabilities
- Use minimal base images
- Rescan after updates

INFRASTRUCTURE SCANNING

- Scan IaC (Terraform, CloudFormation, K8s, etc.)
- Find misconfigurations
- Prevent insecure deployments

KUBERNETES SECURITY CONTEXT

- Set securityContext in all Pods
- Drop capabilities
- Read-only root filesystem

NON-ROOT CONTAINERS

- Run containers as non-root
- Set runAsUser
- Reduce impact of compromise

NETWORK POLICIES

- Control Pod-to-Pod traffic
- Allow only required communication
- Default deny is safer

TLS EVERYWHERE

- Encrypt data in transit
- Enforce TLS 1.2 or higher
- Use valid certificates

ENCRYPTION AT REST

- Encrypt databases, volumes, backups
- Use KMS
- Protect data at all layers

RBAC

- Use Role-Based Access Control
- Grant least privilege access
- Review bindings regularly

AUDITABILITY

- Enable audit logs
- Monitor activities
- Centralize logs
- Detect and respond to anomalies

SOFTWARE SUPPLY CHAIN AWARENESS

- Know your dependencies
- Verify sources and signatures
- Use trusted registries
- Monitor for new vulnerabilities

TEST YOUR CONTROLS

ATTEMPT AN UNAUTHORIZED OPERATION → **VERIFY THAT SECURITY CONTROLS BLOCK IT**

EXAMPLES

- ✓ Access a restricted resource
- ✓ Escalate privileges
- ✓ Exfiltrate secrets
- ✓ Deploy a privileged pod

SECURITY IS EVERYONE'S RESPONSIBILITY.
BUILD SECURE. DEPLOY SECURE. RUN SECURE.

17. BUILD ZERO-DOWNTIME DEPLOYMENT AND ROLLBACK

1. DEPLOYMENT STRATEGIES

ROLLING DEPLOYMENT

Replace old instances with new ones gradually. No downtime.

BLUE-GREEN DEPLOYMENT

Run two identical environments. Switch traffic to Green in one step. Instant rollback to Blue.

CANARY DEPLOYMENT

Send a small % of traffic to the new version first. Increase gradually if healthy.

2. TRAFFIC AND HEALTH

TRAFFIC SHIFTING
Move traffic between versions using ALB, Ingress, or Service Mesh.

HEALTH VERIFICATION
Check readiness, liveness, and external health endpoints.

DEPLOYMENT METRICS
Monitor latency, throughput, CPU, memory, and requests.

ERROR-RATE COMPARISON
Compare error rates between new and old versions.

3. ROLLBACK AUTOMATION

AUTOMATED ROLLBACK CRITERIA
Rollback when error rate, latency, or failed health checks exceed set thresholds.

ROLLBACK APPLICATION VERSION
Revert to the previous stable version automatically.

ROLL-FORWARD STRATEGY
If rollback is not possible, quickly deploy a fixed version forward.

4. DATA AND COMPATIBILITY

DATABASE COMPATIBILITY
Ensure schema changes are backward and forward compatible.

BACKWARD-COMPATIBLE CHANGES
New releases must work with old data and old services.

FEATURE FLAGS
Hide new features behind flags. Enable gradually and safely.

5. PRACTICAL ZERO-DOWNTIME WORKFLOW

1. DEPLOY
Deploy a deliberately faulty release.

2. DETECT
Detect degradation using metrics and alerts.

3. ANALYZE
Confirm the issue (metrics, logs, traces).

4. DECIDE
Decide: Rollback or Roll-forward.

5. RESTORE
Restore service safely with zero or minimal impact.

BEST PRACTICES

- Always verify health before shifting more traffic.
- Automate rollbacks to reduce human delay.
- Use small increments in canary deployments.
- Keep deployments idempotent and repeatable.
- Test rollback process regularly.
- Monitor user experience, not just infrastructure.

ZERO-DOWNTIME CHECKLIST

- Choose deployment strategy (Rolling / Blue-Green / Canary)
- Verify health and metrics
- Check database and backward compatibility
- Enable feature flags
- Monitor error rate, latency, and throughput
- Have automated rollback in place

GOAL: Deliver new versions continuously while keeping users happy.
Detect problems fast. Recover automatically. Keep services available.

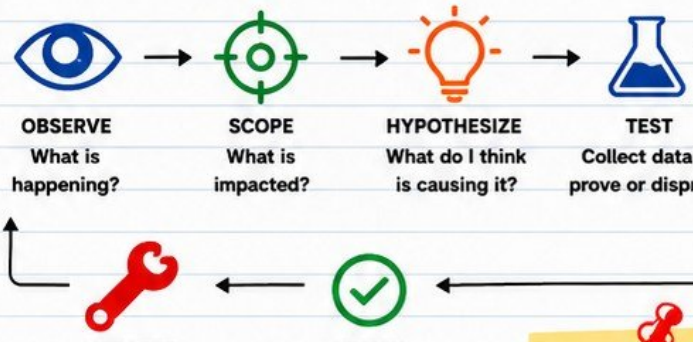
BREAK PRODUCTION ON PURPOSE

BUILD RESILIENCE. FIND WEAKNESSES. IMPROVE BEFORE USERS DO.

CONTROLLED FAILURE SCENARIOS

1. POD CRASH
2. NODE FAILURE
3. CPU SATURATION
4. MEMORY EXHAUSTION
5. DISK PRESSURE
6. DNS FAILURE
7. EXPIRED SECRET
8. DATABASE CONNECTION EXHAUSTION
9. BROKEN SECURITY GROUP
10. BAD KUBERNETES CONFIGURATION
11. FAILED DEPLOYMENT
12. QUEUE BACKLOG
13. DEPENDENCY TIMEOUT
14. HIGH APPLICATION LATENCY
15. HTTP 5XX SPIKE

THE INCIDENT INVESTIGATION FLOW



GOAL

Restore service, understand root cause, and prevent recurrence.

FOR EVERY INCIDENT, CAPTURE THIS

- TIMELINE (WHAT HAPPENED AND WHEN)
- IMPACT (WHO WAS AFFECTED)
- DETECTION METHOD
- OBSERVATIONS AND EVIDENCE
- COMMANDS RUN
- ROOT CAUSE
- MITIGATION TAKEN
- LESSONS LEARNED

USEFUL DIAGNOSTIC COMMANDS

WHAT TO CHECK	COMMAND EXAMPLE
POD STATUS	kubectl get pods -A
POD DETAILS	kubectl describe pod <pod-name> -n <ns>
POD LOGS	kubectl logs <pod-name> -n <ns>
NODE STATUS	kubectl get nodes
NODE ISSUES	kubectl describe node <node-name>
EVENTS	kubectl get events -A --sort-by=.metadata.creationTimestamp
RESOURCE USAGE	kubectl top pods -A
STORAGE	df -h
NETWORK	kubectl get svc,ing,ep -A
DNS	nslookup <service-name> dig <domain>
SECRETS	kubectl get secrets -A
DEPLOYMENTS	kubectl rollout status deployment/<name> -n <ns>

FAILURE LAB EXAMPLE

EXAMPLE: POD CRASH LOOP

1. **BREAK IT:** Update deployment with invalid config or wrong image.
2. **OBSERVE:** kubectl get pods -n <ns> (See CrashLoopBackOff)
3. **SCOPE:** Which service is affected? Check metrics and logs.
4. **HYPOTHESIZE:** Wrong image? Bad config? Missing secret?
5. **TEST:** Check logs, describe pod, events.
6. **MITIGATE:** Rollback or fix configuration.
7. **VERIFY:** Pods running? Metrics normal? Users not impacted?

TIP:
Break things in lower environments first.
Document everything as if it were real.

REMEMBER

- FAIL FAST IN SAFE ENVIRONMENTS.
- AUTOMATE VALIDATION.
- BUILD SELF-HEALING SYSTEMS.
- OBSERVABILITY FIRST.
- DOCUMENT YOUR RECOVERY.
- TURN INCIDENTS INTO IMPROVEMENTS.

REAL ENGINEERS DON'T FEAR FAILURES. THEY USE THEM TO BUILD RELIABLE SYSTEMS.

RUN A REAL INCIDENT AND WRITE THE POSTMORTEM

NO BLAME. EVIDENCE FIRST.

SCENARIO

A DEPLOYMENT SUCCEEDS.

FIVE MINUTES LATER:

- HTTP 5xx INCREASES
- P95 LATENCY RISES
- SOME PODS RESTART
- DATABASE CONNECTIONS INCREASE
- CUSTOMERS REPORT INTERMITTENT FAILURES

SOMETHING IS WRONG. LET'S INVESTIGATE.

INVESTIGATE

- DEPLOYMENT HISTORY
- KUBERNETES EVENTS
- POD LOGS
- METRICS
- TRACES
- DATABASE STATE
- RECENT CONFIGURATION CHANGES

GATHER EVIDENCE BEFORE FORMING CONCLUSIONS.

START WITH OBSERVABILITY

METRICS
Check error rate, latency, traffic, CPU, memory, DB connections.

LOGS
Look for errors, exceptions, timeouts, restarts.

TRACES
Follow a request end-to-end.

DATABASE
Connections, slow queries, locks, resource usage.

CONFIG CHANGES
Recent deployments, feature flags, secrets, config updates.

INVESTIGATION FLOW (FOLLOW THIS LOOP)

1. **OBSERVE** — What is happening?
2. **SCOPE** — What is impacted?
3. **HYPOTHESIZE** — What could be causing it?
4. **TEST** — Collect data to prove or disprove.
5. **MITIGATE** — Apply fix or workaround.
6. **VERIFY** — Confirm the issue is resolved.

POSTMORTEM: WHAT TO PRODUCE

- **INCIDENT TIMELINE** — A clear timeline of what happened and when.
- **CUSTOMER IMPACT** — Who was affected and how.
- **DETECTION METHOD** — How was the incident detected?
- **ROOT CAUSE** — The primary reason the incident happened.
- **CONTRIBUTING FACTORS** — Other factors that made the incident worse.
- **MITIGATION** — What actions reduced the impact?
- **RECOVERY** — How was full service restored?
- **CORRECTIVE ACTIONS** — What will we fix in the system?
- **PREVENTIVE ACTIONS** — What will we do to avoid this in the future?
- **LESSONS LEARNED** — What did we learn as a team?

EVIDENCE FIRST

- NO ASSUMPTIONS
- NO BLAME
- NO GUESSING
- JUST FACTS
- JUST IMPACT
- JUST IMPROVEMENT

PRO TIP

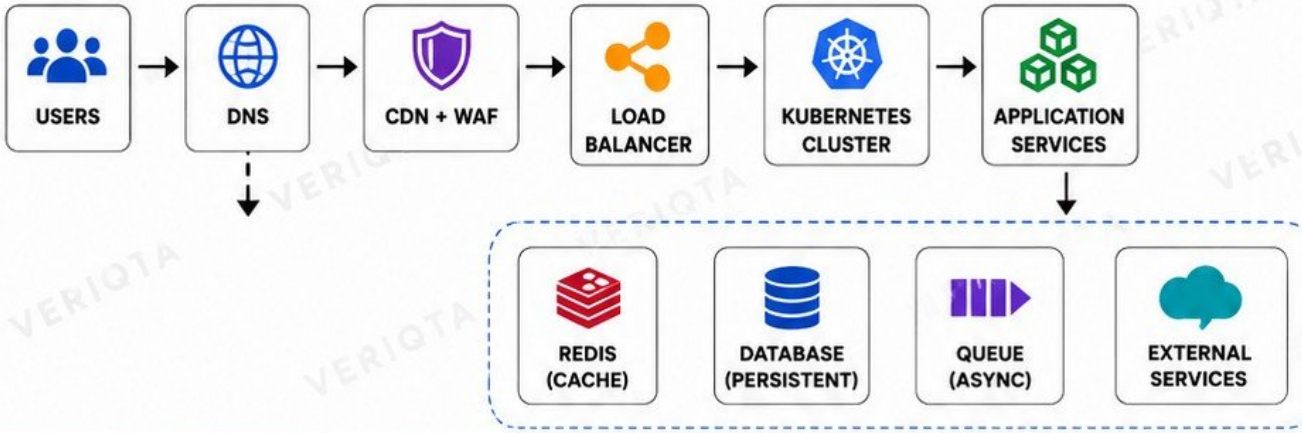
WRITE POSTMORTEMS WHILE IT'S FRESH. ACCURACY MATTERS. CLARITY HELPS EVERYONE IMPROVE.

THE GOAL IS NOT TO FIND SOMEONE TO BLAME.
THE GOAL IS TO IMPROVE THE SYSTEM SO IT DOESN'T HAPPEN AGAIN.

FINAL PRODUCTION PLATFORM PROJECT

BUILD. DELIVER. OPERATE. IMPROVE.

END-TO-END SYSTEM ARCHITECTURE



INFRASTRUCTURE (AS CODE)

GIT → TERRAFORM → aws

MANAGED WITH INFRASTRUCTURE AS CODE

- REPRODUCIBLE
- VERSION CONTROL
- REVIEWABLE
- AUDITABLE
- REPEATABLE
- DRIFT FREE

DELIVERY PIPELINE

PULL REQUEST → CI (BUILD & TEST) → SECURITY CHECKS → CONTAINER REGISTRY → STAGING → PRODUCTION APPROVAL → KUBERNETES DEPLOYMENT → VERIFICATION

DELIVERY PRINCIPLES

- AUTOMATED
- REPEATABLE
- SAFE
- TRACEABLE
- FAST FEEDBACK
- ROLLBACK READY

OPERATIONS & OBSERVABILITY

PROMETHEUS (METRICS) · GRAFANA (DASHBOARDS) · LOGS (CENTRALIZED) · TRACES (DISTRIBUTED)

→ CLOUD MONITORING (ALERTS & EVENTS)

WHAT YOU CAN SEE

- HEALTH
- LATENCY
- ERROR RATE
- TRAFFIC
- SATURATION
- DEPENDENCIES
- DATABASE
- QUEUES
- COST
- SECURITY EVENTS

FINAL REQUIREMENTS (PRODUCTION READINESS CHECKLIST)

- INFRASTRUCTURE AS CODE
- MULTI-AZ ARCHITECTURE
- AUTOMATED CI/CD
- SECURE SECRET HANDLING
- CONTAINER SECURITY
- HIGH AVAILABILITY
- AUTOSCALING
- MONITORING
- ALERTING
- BACKUP AND RECOVERY
- ZERO-DOWNTIME DEPLOYMENT
- ROLLBACK
- FAILURE TESTING
- INCIDENT RESPONSE
- ARCHITECTURE DOCUMENTATION
- RUNBOOK
- POSTMORTEM PROCESS
- FINAL PRODUCTION REVIEW

PRODUCTION IS NOT A DESTINATION. IT IS A DISCIPLINE.

CONTINUOUSLY MEASURE. CONTINUOUSLY IMPROVE. CONTINUOUSLY PROTECT.

THE FINAL QUESTION IS NOT:

"DOES THE APPLICATION RUN?" →

THE REAL QUESTION IS:

"CAN ANOTHER ENGINEER SAFELY DEPLOY, OBSERVE, TROUBLESHOOT, RECOVER, AND OPERATE THIS SYSTEM WHEN THINGS GO WRONG?"